Understanding and Debugging DNS in Kubernetes Clusters

Cloud Native Rejekts Europe 2025

About me

- Pakistan (Lahore) -> Germany (Hamburg)
- Software Engineer @ Microsoft
- Inspektor Gadget, CoreDNS header plugin, other cloud-native projects!

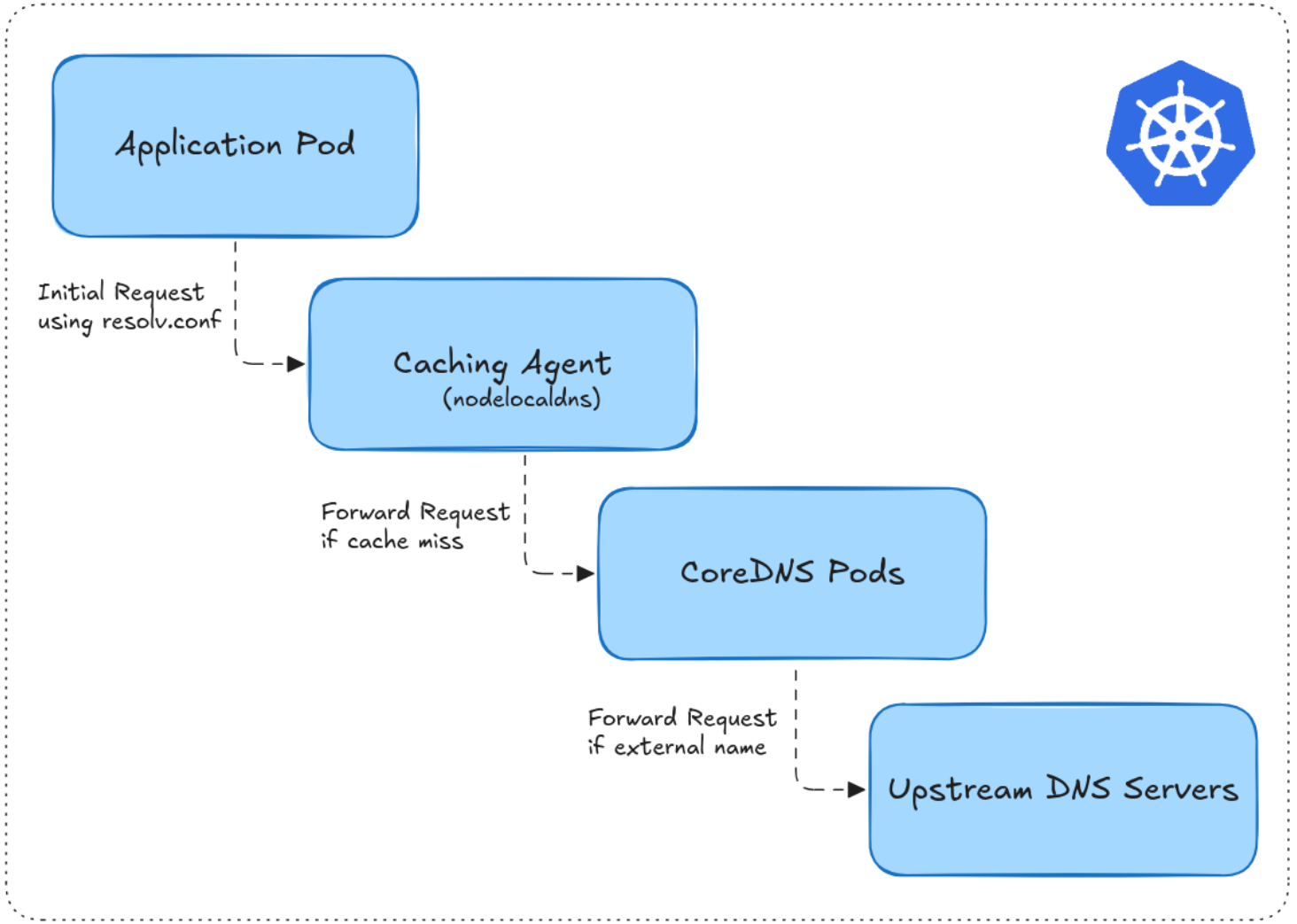
Agenda

1. Understanding DNS Components / Requests Flows
2. Deep Dive into Request Flows using:
   - CoreDNS log plugin
   - Hubble
   - Inspektor Gadget
3. Debugging Scenarios

Understanding DNS Components / Request Flows

- To debug DNS issues, we first need to understand different components:
  1. Application Pod
  2. Node Local DNS
  3. kube-dns Service
  4. CoreDNS Pod
  5. Upstream DNS Server

- Challenges:
  - Too many hidden systems.
  - Not easy to trace the DNS request flows across the cluster.
  - Which tools to use to deep dive into request flows?



Deep Dive into Request Flows

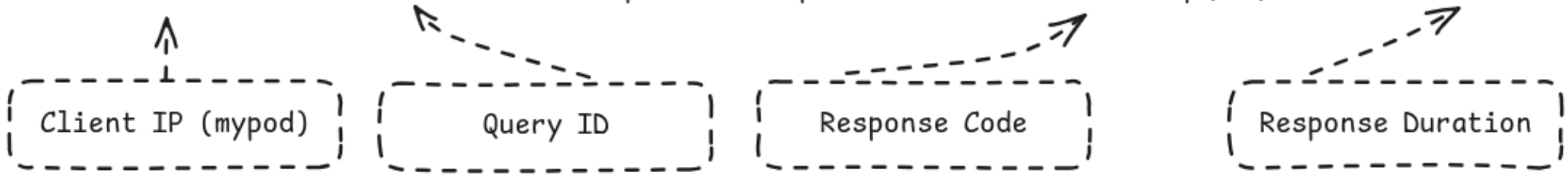
CoreDNS log plugin

- CoreDNS core plugin: https://coredns.io/plugins/log/
- Needs a keyword log in Corefile.
- Logs all the requests to stdout.

```
log [NAMES...] [FORMAT] {
    class CLASSES...
}
```

CoreDNS log plugin

```
$ kubectl logs -n kube-system -l k8s-app=kube-dns -f
[INFO] 10.244.0.173:34432 - 45551 "A IN example.com. udp 29 false 512" NOERROR qr,rd,ra 56 0.021960059s
```

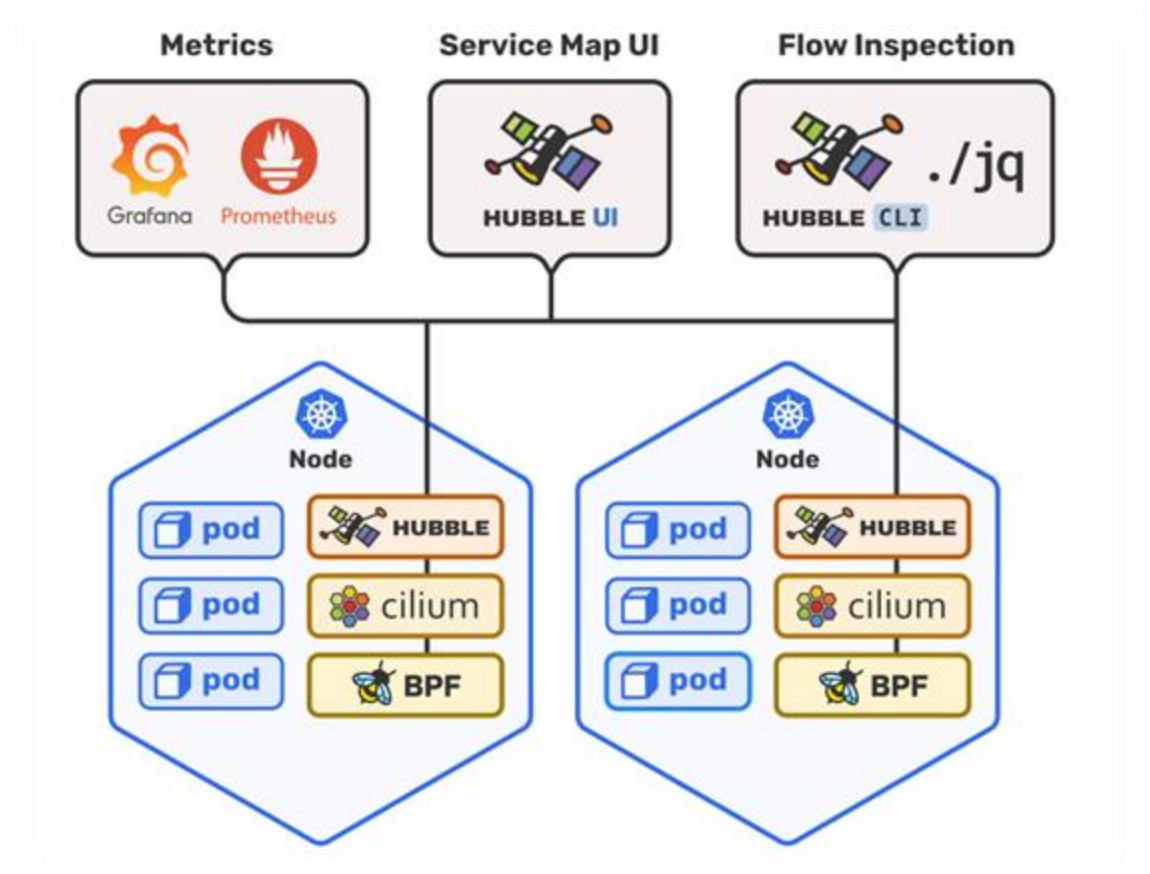


Log Format:

```
{remote}:{port} - {>id} "{type} {class} {name} {proto} {size} {>do} {>bufsize}" {rcode} {>rflags} {rsize} {duration}
```

Hubble

- Provides Network, Service & Security Observability for Kubernetes.
- Built on top of Cilium and eBPF.
- Installed as Kubernetes Daemonset.



source: https://github.com/cilium/hubble

Hubble

- Install Cilium with L7 Proxy Support.
- Enable DNS traffic visibility using CiliumNetworkPolicy for:
  - mypod
  - CoreDNS

```yaml
apiVersion: cilium.io/v2
kind: CiliumNetworkPolicy
metadata:
  name: mypod-visibility
  namespace: demo
spec:
  endpointSelector:
    matchLabels:
      run: mypod
  egress:
  - toEntities:
    - all
  - toEndpoints:
    - matchLabels:
        k8s:io.kubernetes.pod.namespace: kube-system
        k8s:k8s-app: kube-dns
    toPorts:
    - ports:
      - port: "53"
        protocol: ANY
      rules:
        dns:
        - matchPattern: '*'
```

mypod

```yaml
apiVersion: cilium.io/v2
kind: CiliumNetworkPolicy
metadata:
  name: kube-dns-visibility
  namespace: kube-system
spec:
  endpointSelector:
    matchLabels:
      k8s-app: kube-dns
  egress:
  - toEntities:
    - all
  - toPorts:
    - ports:
      - port: "53"
        protocol: ANY
      rules:
        dns:
        - matchPattern: '*'
```

CoreDNS

Hubble

```
$ hubble observe --protocol dns -f
Aug 26 13:03:59.569: demo/mypod:49105 (ID:39877) -> kube-system/coredns-6948557899-m7d49:53 (ID:22818) dns-request proxy FORWARDED (DNS Query example.com. A)
Aug 26 13:03:59.570: kube-system/coredns-6948557899-m7d49:43347 (ID:22818) -> 192.168.49.1:53 (world) dns-request proxy FORWARDED (DNS Query example.com. A)
Aug 26 13:03:59.572: kube-system/coredns-6948557899-m7d49:43347 (ID:22818) <- 192.168.49.1:53 (world) dns-response proxy FORWARDED (DNS Answer "93.184.215.14" TTL: 3091 (Proxy example.com. A))
Aug 26 13:03:59.572: demo/mypod:49105 (ID:39877) <- kube-system/coredns-6948557899-m7d49:53 (ID:22818) dns-response proxy FORWARDED (DNS Answer "93.184.215.14" TTL: 30 (Proxy example.com. A))
```

Filtering:

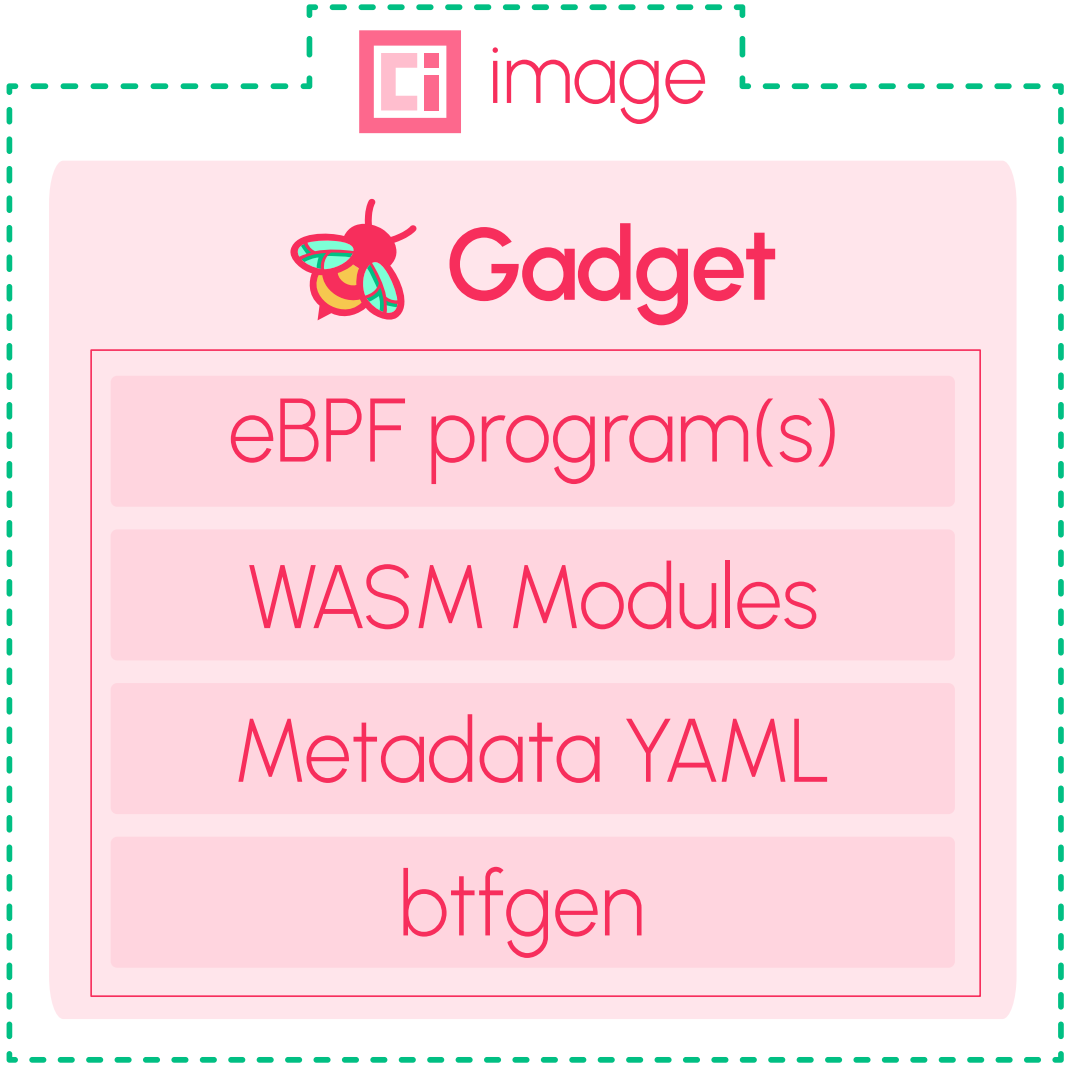
```
$ nslookup -query=a unknown.example.com.
$ hubble observe --since=5m --protocol dns -o json \
  | jq 'select(.flow.l7.dns.rcode==3) | .flow.destination.namespace + "/" + .flow.destination.pod_name' \
  | sort | uniq -c | sort -r
    1 "kube-system/coredns-6948557899-m7d49"
    1 "demo/mypod"
```

Inspektor Gadget

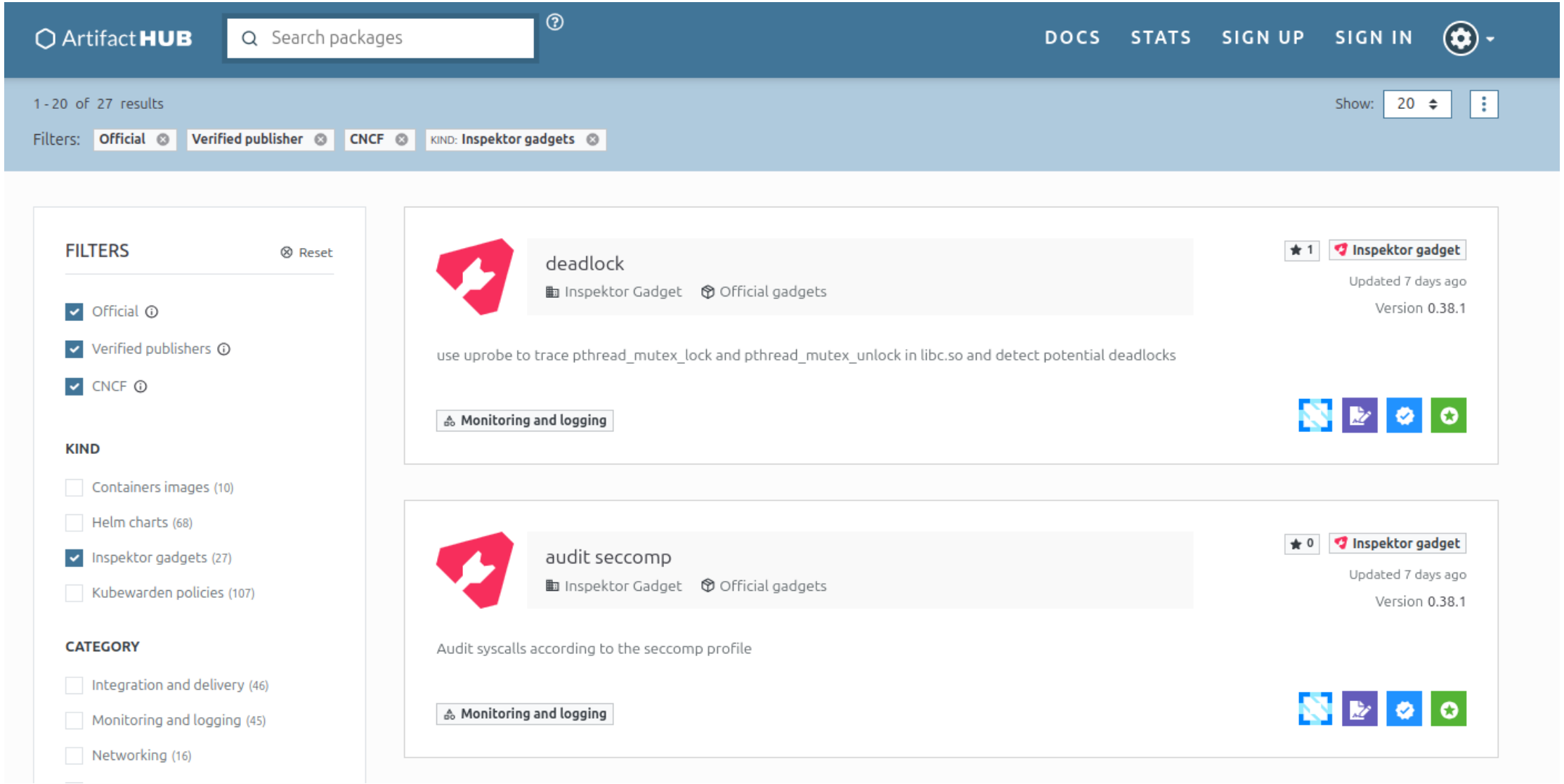
- **Tool:** A set of tools (gadgets) that empower users to inspect Kubernetes and Linux Systems.
- **Framework:** An observability framework that allows you to build and share custom "gadgets".
- It can be used in Kubernetes as:
  - Daemonset + kubectl gadgets.
  - kubectl debug node.

Inspektor Gadget – Gadget

- A gadget is a sharable and runnable unit for debugging and observability.
- For example:
  - Trace DNS events.
  - Examine drop reasons for TCP connections.
  - Watch which processes are being executed in your cluster.



Inspektor Gadget - Gadget



Inspektor Gadget – DNS Gadget

- DNS Gadget:
  - **kubectl gadget run trace_dns**
  - https://inspektor-gadget.io/docs/latest/gadgets/trace_dns
  - https://github.com/inspektor-gadget/inspektor-gadget/tree/main/gadgets/trace_dns
- Trace DNS requests and responses using eBPF.
- Uses WASM for post-processing.
- Source code. (Modify -> Build -> Push -> Run)

Source Code

Inspektor Gadget (Application Pod)

```
$ kubectl gadget run -f https://raw.githubusercontent.com/mqasimsarfraz/talks/refs/heads/main/CloudNativeRejekts-2025/InspektorGadget/instance-manifest/application-pod.yaml
K8S.PODNAME  SRC                      DST                      NAME          QR RCODE
mypod        p/demo/mypod:36296       s/kube-system/kube-dns:53 example.com.  Q
mypod        s/kube-system/kube-dns:53 p/demo/mypod:36296       example.com.  R  Success
```

Inspektor Gadget (Application Pod)

main talks / CloudNativeRejekts-2025 / InspektorGadget / instance-manifest / **application-pod.yaml**


mqasimsarfraz Add CloudNativeRejekts-2025

Code Blame 24 lines (24 loc) · 1.06 KB

```yaml
# Trace DNS requests of the application pod in the demo namespace
# Gadgets used:
# - trace_dns (https://www.inspektor-gadget.io/docs/latest/gadgets/trace_dns)
apiVersion: 1
kind: instance-spec
image: trace_dns:v0.38.1
name: application-pod
paramValues:
  # The following parameter is used to trace DNS requests in the default namespace
  # See: https://www.inspektor-gadget.io/docs/latest/spec/operators/kubemanager
  operator.KubeManager.namespace: demo
  # The following parameter is used to filter the DNS requests that are:
  #   1. IPv4 requests
  #   2. Domain name is example.com
  # See: https://www.inspektor-gadget.io/docs/latest/spec/operators/filter
  operator.filter.filter: qtype==A,name==example.com.
  # The following parameter is used to display the following fields in the output:
  #   1. Name of the pod
  #   2. Source endpoint of the DNS request
  #   3. Destination endpoint of the DNS request
  #   4. Name in the DNS request
  #   5. Query type of the DNS request
  #   6. Response code of the DNS request
  operator.cli.fields: k8s.podname,src,dst,name,qr,rcode
```

Inspektor Gadget (Application + CoreDNS Pods)

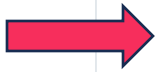
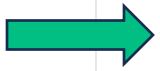
```
$ kubectl gadget run -f https://raw.githubusercontent.com/mqasimsarfraz/talks/refs/heads/main/CloudNativeRejekts-2025/InspektorGadget/instance-manifest/application-coredns-pod-latency.yaml
K8S.PODNAME                SRC                          DST                          NAME          ID    QR RCODE    LATENCY_NS
mypod                      p/demo/mypod:52706           s/kube-system/kube-dns:53    example.com.  cd51  Q           0ns
coredns-7db6d8ff4d-jnqlx   p/demo/mypod:52706           p/kube-system/coredns-7db6d8ff4...  example.com.  cd51  Q           0ns
coredns-7db6d8ff4d-jnqlx   p/kube-system/coredns-7db6d8ff4...  192.168.49.1:53       example.com.  0892  Q           0ns
coredns-7db6d8ff4d-jnqlx   192.168.49.1:53              p/kube-system/coredns-7db6d8ff4...  example.com.  0892  R  Success  22.78ms
coredns-7db6d8ff4d-jnqlx   p/kube-system/coredns-7db6d8ff4...  p/demo/mypod:52706    example.com.  cd51  R  Success  0ns
mypod                      s/kube-system/kube-dns:53    p/demo/mypod:52706           example.com.  cd51  R  Success  23.17ms
```

Inspektor Gadget (Application + CoreDNS Pods)

main talks / CloudNativeRejekts-2025 / InspektorGadget / instance-manifest / application-coredns-pod-latency.yaml

mqasimsarfraz Add CloudNativeRejekts-2025

Code Blame 26 lines (26 loc) · 1.19 KB

```
1   # Trace DNS requests in demo and kube-system namespaces
2   # Gadgets used:
3   # - trace_dns (https://www.inspektor-gadget.io/docs/latest/gadgets/trace_dns)
4   apiVersion: 1
5   kind: instance-spec
6   image: trace_dns:v0.38.1
7   name: application-coredns-pod-latency
8   paramValues:
9     # The following parameter is used to trace DNS requests in the default namespace
10    # See: https://www.inspektor-gadget.io/docs/latest/spec/operators/kubemanager
11    operator.KubeManager.namespace: demo,kube-system
12    # The following parameter is used to filter the DNS requests that are:
13    #   1. IPv4 requests
14    #   2. Domain name is example.com
15    # See: https://www.inspektor-gadget.io/docs/latest/spec/operators/filter
16    operator.filter.filter: k8s.podName~mypod|coredns-*,qtype==A,name==example.com.
17    # The following parameter is used to display the following fields in the output:
18    #   1. Name of the pod
19    #   2. Source endpoint of the DNS request
20    #   3. Destination endpoint of the DNS request
21    #   4. Name in the DNS request
22    #   5. Query type of the DNS request
23    #   6. ID of the DNS request
24    #   7. Response code of the DNS request
25    #   8. Latency of the DNS request
26    operator.cli.fields: k8s.podname,src,dst,name,id,qr,rcode,latency_ns
```

Summary

CoreDNS log plugin	Hubble	Inspektor Gadget
Ideal for initial inspection; however; its scope it limited to CoreDNS.	Provide compact overview of request flows with enrichment and retrospective analysis	Offers rich DNS traces with Kubernetes enrichment/filtering.
Requires configuration changes for use.	Requires Cilium CNI / specific policies to be in place for L7 flow visibility	Extensible through custom gadget images ; Limited TCP support.

Debugging Scenarios

Debugging Scenario 1: Verify Health of an Upstream DNS Server

Debugging Scenario 2: Identify slow DNS requests in the cluster.

Debugging Scenario 3: Tracing DNS requests using Ephemeral Containers.

Conclusion

- First step for debugging DNS is understanding the request flows and:
  - CoreDNS log plugin
  - Hubble
  - Inspektor Gadget
- This was based on my personal debugging workflow, so feedback is appreciated.
- Finally, reach out respective communities if you have any questions/suggestions.

Inspektor Gadget at KubeCon

Questions?

Slides